

Competitive Programming

Section 50 Editorial: Advanced Algorithms Mixed Contest

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. DAG Grundy	4
2.1	Restatement	4
2.2	Algorithm and proof	4
2.3	Complexity	4
2.4	Full C++ solution	4
2.5	Full Python solution	4
3	B. General CRT	5
3.1	Restatement	5
3.2	Algorithm and proof	5
3.3	Complexity	5
3.4	Full C++ solution	5
3.5	Full Python solution	5
4	C. Fast Fibonacci	7
4.1	Restatement	7
4.2	Algorithm and proof	7
4.3	Complexity	7
4.4	Full C++ solution	7
4.5	Full Python solution	7
5	Testing discussion	8

1 Editorial overview

1.1 Detailed solution guide

A. DAG Game Grundy. Process vertices in reverse topological order. The Grundy value of a terminal vertex is 0. For a nonterminal vertex, collect the Grundy values of all outgoing neighbors and take the mex, the smallest nonnegative integer not present. A position is winning exactly when its Grundy value is nonzero: move to a child with Grundy 0; from Grundy 0, every move goes to nonzero.

B. Chinese Remainder Merge. To merge $x = a \pmod m$ and $x = b \pmod n$, first check compatibility: $a \% \gcd(m,n)$ must equal $b \% \gcd(m,n)$. If not, there is no solution. If yes, divide by the gcd and use an extended-gcd inverse to solve for the offset. The new modulus is $\text{lcm}(m,n)$, and the merged residue is normalized into $[0, \text{lcm})$.

C. Fast Doubling Fibonacci. Maintain the pair $(F(k), F(k+1))$. The doubling formulas compute $(F(2k), F(2k+1))$; if the next binary bit is 1, advance once to $(F(2k+1), F(2k+2))$. This gives logarithmic time because each bit of n does constant work.

Common pitfalls. For Grundy values, mex is taken over children only, not all smaller numbers. CRT must handle non-coprime moduli. Fast doubling should keep all operations modulo the requested modulus.

This mixed section checks whether the model is recognized quickly.

- A computes Grundy numbers on a DAG in reverse topological order. A start is winning iff its Grundy number is nonzero.
- B merges two congruences with the generalized CRT. The compatibility test is equality modulo $\gcd(m,n)$.
- C uses fast doubling for Fibonacci, returning the pair $(F(n), F(n+1))$.

All three solutions compress a large object into a smaller invariant: mex, one merged congruence, or a recurrence pair.

2 A. DAG Grundy

2.1 Restatement

For each start vertex in a DAG, print whether the first player wins.

2.2 Algorithm and proof

Correctness follows from the problem-specific reduction in the detailed guide: the generated candidates or states cover exactly the legal choices, and the final query applies the required count, minimum, maximum, merge, or classification. \square

2.3 Complexity

The complexity is the one explained in the detailed guide and follows from the number of generated states plus the searches, transitions, or graph traversals used below.

2.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m,q;if(!
```

2.5 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m,q=data[0],data[1],data[2]; idx=3; g=[] for _ in range(n)
    for _ in range(m): g[data[idx]-1].append(data[idx+1]-1); idx+=2
    gr=[0]*n
    for u in range(n-1,-1,-1):
        seen={gr[v] for v in g[u]}; x=0
        while x in seen: x+=1
        gr[u]=x
    out=["WIN" if gr[data[idx+i]-1] else "LOSE" for i in range(q)]
    print("\n".join(out))
if __name__=="__main__": main()
```

3 B. General CRT

3.1 Restatement

For each pair of congruences, print the smallest nonnegative solution and lcm modulus, or NO.

3.2 Algorithm and proof

Correctness follows from the problem-specific reduction in the detailed guide: the generated candidates or states cover exactly the legal choices, and the final query applies the required count, minimum, maximum, merge, or classification. \square

3.3 Complexity

The complexity is the one explained in the detailed guide and follows from the number of generated states plus the searches, transitions, or graph traversals used below.

3.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; long long eg(long long a,long long b,long long&x,long long&y){if(!b){x=
```

3.5 Full Python solution

```
import sys,math
def eg(a,b):
    if b==0: return (abs(a),1 if a>=0 else -1,0)
    g,x,y=eg(b,a%b); return g,y,x-(a//b)*y
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    t=data[0]; idx=1; out=[]
    for _ in range(t):
        a,m,b,n=data[idx],data[idx+1],data[idx+2],data[idx+3]; idx+=4; g,x,y=eg(m,n)
        if (b-a)%g: out.append("NO")
        else:
            l=m//g*n; k=((b-a)//g*x%(n//g)); out.append(f"{(a+m*k)%l} {l}")
```

```
    print("\n".join(out))  
if __name__=="__main__": main()
```

4 C. Fast Fibonacci

4.1 Restatement

Answer Fibonacci queries modulo $1e9+7$.

4.2 Algorithm and proof

Correctness follows from the problem-specific reduction in the detailed guide: the generated candidates or states cover exactly the legal choices, and the final query applies the required count, minimum, maximum, merge, or classification. `\square`

4.3 Complexity

The complexity is the one explained in the detailed guide and follows from the number of generated states plus the searches, transitions, or graph traversals used below.

4.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; const long long MOD=1000000007; pair<long long,long long> fib(long long
```

4.5 Full Python solution

```
import sys
MOD=1_000_000_007
def fib(n):
    if n==0: return (0,1)
    a,b=fib(n//2)
    c=a*((2*b-a)%MOD)%MOD
    d=(a*a+b*b)%MOD
    return (d,(c+d)%MOD) if n%2 else (c,d)
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    print("\n".join(str(fib(x)[0]) for x in data[1:1+data[0]]))
if __name__=="__main__": main()
```

5 Testing discussion

Random tests compare C++ and Python against fixed expected outputs generated from the reference implementation. Important cases include singletons, negative numbers where allowed, repeated values, and boundary query sizes.